Java, Hardware & Programming Languages

Short answers: Java does not talk to hardware directly, and PC drivers are not made with Java.

Why Java?

Java trades hardware access for portability and safety. Your code compiles to bytecode, which runs on the JVM (Java Virtual Machine), not the CPU directly. The JVM handles memory, checks array bounds, and garbage-collects. Result: 'write once, run anywhere,' fewer crashes, good for backends, Android apps, and enterprise systems. Cost: slower startup, more memory, no direct hardware control.

What actually touches hardware?

Bottom-up layers:
- Machine code / assembly: what the CPU executes. Nobody writes drivers in this by hand anymore.
- C: the driver/OS language. Compiles straight to machine code, manual memory, pointer access to hardware registers. Windows/Linux kernels and nearly all drivers are C.
- C++: C plus classes/templates. Used in drivers, game engines, browsers.
- Rust: newer systems language, same low-level control as C++ but the compiler rejects memory bugs. Now allowed in the Windows and Linux kernels for new driver code.

Other languages and how they work

- Managed/compiled: C# (.NET runtime, like the JVM), Go (compiles to native, garbage-collected; servers and cloud tools).
- JIT/bytecode: Java, Kotlin (JVM); JavaScript/TypeScript (V8 engine JITs to machine code in the browser and Node).
- Interpreted/scripting: Python, Ruby: an interpreter executes them line by line (CPython). Easy to write, roughly 10-50x slower than C; heavy work gets pushed to C extensions (NumPy, etc.).
- Functional/niche: Haskell, Erlang/Elixir (massively concurrent telecom-style systems), Swift (Apple apps), SQL (databases; declarative, not general-purpose).

Rule of thumb

Closer to hardware = C/C++/Rust (fast, dangerous, manual memory). Farther = Java/C#/Go/Python/JS (safe, portable, slower, runtime manages memory). Drivers live at the bottom layer, so they are C, C++, and increasingly Rust - never Java.